

6팀 중급 프로젝트 종합보고서

프로젝트명 It's mine (B2G 입찰지원 RFP 분석 서비스)

팀원 유수빈 · 윤승준 · 정승호 · 정진우

역할별 담당 업무

이름	역할	담당 업무
윤승준	PM (Project Manager)	서비스 기획과 프로젝트 방향 설정, 팀 협업 관리
정승호	DE (Data Engineer)	원본 데이터 및 메타데이터 검증, 평가용 질문셋 제작
유수빈	EL (Experiment Lead)	RAG 파이프라인 실험 설계 및 실행, 모델 성능 고도화 작업, 실험 성능 지표 비교
정진우	PL (Presentation Lead)	웹 서비스 구현(프론트엔드), 데모 앱 스택 조사, 최종 발표, 보고서 준비

목차

- 1. 프로젝트 개요
 - 2. 전체 작업 흐름 한눈에 보기
 - 3. 데이터 준비: 원본 검증부터 텍스트 복원까지
 - 4. 청킹 파이프라인 설계와 개선
 - 5. 평가용 골든 데이터셋 구축
 - 6. RAG 실험 및 하이퍼파라미터 튜닝
 - 7. 하이퍼파라미터 전후 답변 비교
 - 8. Vector DB 선정
 - 9. 프론트엔드(웹 UI) 구현
 - 10. 백엔드 서비스 구현 및 고도화 계획
 - 11. 서비스 발전 방향
 - 12. 한계 및 남은 과제
 - 13. 부록: 버전 계보 및 핵심 수치
-

1. 프로젝트 개요

1.1 배경과 문제 정의

공공조달 시장에서는 나라장터를 비롯한 조달 포털에 하루에도 수백 건의 RFP(제안요청서, Request for Proposal)가 올라옵니다. 한 건당 수십 페이지에 달하는 문서를 기업 담당자가 일일이 읽어 예산·발주기관·참가자격·제출서류·평가기준 같은 핵심 정보를 파악하는 것은 시간이 많이 드는 일입니다. **It's mine**은 이 문제를 RAG(Retrieval-Augmented Generation) 기반으로 해결하는 서비스로, 사용자가 선택한 RFP 문서를 요약하고 핵심 요구사항을 추출·질의응답할 수 있게 하는 것을 목표로 합니다.

프로젝트는 B2G 입찰지원 컨설팅 스타트업의 사내 RAG 시스템을 구축한다는 시나리오를 전제로 하며, 실제 공공조달 RFP 문서 100건과 각 문서의 메타데이터(공고번호, 사업명, 발주기관, 사업금액, 입찰일정 등)를 제공받아 진행되었습니다.

1.2 목표

- 복잡한 형태의 정부·공공기관 RFP 문서에서 필요한 정보를 정확히 추출·요약하는 Q&A 시스템 구축
- 단일 문서 정보 추출뿐 아니라 여러 문서를 종합한 비교·분석, 후속 질문의 맥락 유지, 문서에 없는 내용에 대한 정직한 "모름" 응답까지 지원
- 팀이 직접 정의한 평가 지표로 성능을 측정하고, 의사결정 과정을 보고서와 발표에 투명하게 드러내기

1.3 기술 스택 개요

구분	채택
임베딩 모델	OpenAI <code>text-embedding-3-small</code>
답변 생성 모델	OpenAI <code>gpt-5-mini</code>
Vector DB	ChromaDB (우선 적용), 확장 시 PostgreSQL + pgvector
프레임워크	LangChain 기반 RAG 엔진
프론트엔드	Streamlit
인프라	GCP VM (g2-standard, L4 GPU), conda 환경

1.4 문서 데이터 특성

제공된 100건은 hwp 96건, pdf 4건으로 구성되었습니다. 이 데이터는 이후 전 과정에서 반복적으로 문제의 원천이 되는데, 그 이유는 다음과 같습니다.

- HWP 바이너리 구조:** 한글(HWP5) 문서는 본문이 표 안에 들어 있는 경우가 많고, 텍스트가 레코드 단위 바이너리로 저장되어 단순 디코딩으로는 온전히 추출되지 않습니다.
- 메타데이터의 문서 외부성:** 발주기관·사업금액·입찰일정 같은 정보는 RFP 본문이 아니라 조달 포털이 부여하는 값인 경우가 많아, 문서를 아무리 읽어도 본문에 없는 경우가 흔합니다.
- 목차·서식 노이즈:** 문서 앞부분의 목차, HWP 페이지번호 특수문자 등이 검색을 방해하는 노이즈로 작용합니다.

이 세 가지 특성이 데이터 준비 단계 전체를 관통하는 핵심 난관이었습니다.

2. 전체 작업 흐름 한눈에 보기

data_list.csv (100건, 원본)

- | 텍스트 500자 미만 7건 = 추출 실패로 진단·격리

▼

HWP5 레코드 기반 파서 신규 개발 → 7건 복원

- | 골든셋 최초 생성(200문항), 중복 2건 제거(100→98행), "마감=개찰-1시간" 오류 수정

▼

data_list_버전3 (98건, 결측 32→17건)

- | 파서가 나머지 91건엔 미적용이던 결 발견 → 전량 재추출, 결측 0건

▼

data_list_버전4 (98건, 텍스트 평균 72,802자) + eval_questions_v4 (344문항)

- | 청킹(16,218개) + chunk_id 매핑, langchain fallback 버그 발견, 첫 청크 메타 프리앰블
- | 정답 verbatim 검증 → 378문항(v6) 확정

▼

data_list_버전5 (목차 제거 + 전 청크 메타 프리앰블 + overlap 80→40, 청크 14,717개)

- | fuzzy 오배정 수정, 유니코드 깨짐 반복 패치, 콘텐츠 오류 수정 → 371문항(v11)

▼

eval_questions_v12_final (질문 제목 26건 수정, 예산 라벨 통일 → 370문항, 최신)

- |

▼

RAG 실험: 임베딩 교체(29.5→78.5%) → 평가체계 정밀화 → reranker → BM25+RRF → top_k=15+reranker

- | 최종: v12mini top15+reranker → retrieval_hit 96%, judge_correct 80%

▼

서비스화: ChromaDB 선정 → 인메모리→DB 전환, 응답 품질/off-topic/속도 고도화

이 흐름은 크게 세 과정으로 나뉩니다.

1. **데이터 준비 과정:** "문서에서 텍스트를 온전히 뽑아내고, 메타데이터를 정확히 채우는" 작업. 파서를 새로 만들고, 결측을 조사하고, 91건을 전량 재추출했습니다.
2. **평가 기반 구축 과정:** "무엇을 정답으로 볼 것인가"를 정의하는 골든셋 구축과, "청크를 어떻게 자를 것인가"를 다듬는 청킹 개선. 이 두 가지가 맞물려 반복 개선되었습니다.
3. **실험·서비스화 과정:** 완성된 데이터·골든셋 위에서 임베딩·retrieval·답변 모델을 튜닝하고, 그 결과를 실제 웹 서비스로 옮기는 작업.

3. 데이터 준비: 원본 검증부터 텍스트 복원까지

3.1 원본 상태 파악과 이슈 문서 격리

원본 `data_list.csv` 는 100건 문서의 메타데이터와 본문 텍스트를 담은 파일로, 컬럼은 세 출처에서 왔습니다.

- **포털 메타데이터**: 공고번호, 공고차수, 공개일자, 입찰 참여 시작일·마감일 (나라장터 등에서 수집)
- **문서 내용**: 사업명, 사업금액, 발주기관, 사업요약
- **파일 관리**: 파일형식, 파일명, 텍스트(추출 본문)

초기 통계는 총 100건, hwp 96 / pdf 4, 텍스트 평균 3,843자, 최대 18,335자, **최소 89자**였습니다. 텍스트 길이 500자 미만 문서 7건을 이슈로 분류했는데, 500자를 기준으로 삼은 이유는 RAG의 기본 `chunk_size` 가 500자여서 문서 전체가 청크 하나보다 짧으면 검색 자체가 무의미하기 때문입니다.

공고번호	사업명	텍스트 길이	원인
20240827859	EIP3.0 고압가스 안전관리 시스템 구축 용역	234자	목차까지만, 본문 누락
20240821865	스포츠윤리센터 LMS 기능개선	298자	목차까지만, 본문 누락
(없음)	학업성취도 다차원 종단분석 (서울시립대, pdf)	220자	1페이지만 추출
20240910050	구미아시아육상경기 종합정보시스템	186자	대회 개요만, 본문 누락
(없음)	건설기술 특허·실용신안 관리시스템	89자	제목+날짜만
20240903688	JST 공유대학 xAPI LRS시스템	130자	표지만 추출
(없음)	사업장 사회보험료 정보시스템 보완 개발	401자	청렴 안내문만

이 7건은 삭제하지 않고 보존하되 평가 질문 생성 대상에서만 제외하는 방식으로 격리했습니다. 이후 93개 정상 문서로 골든셋 317개를 처음 만들었으나 품질 문제(자격 4/10, 제출서류 3/10)를 발견해 폐기하고, LLM 의미 기반 추출을 더한 **897개**로 재작성했습니다. 이 897개가 이후 골든셋의 출발점이 됩니다.

3.2 HWP5 레코드 기반 파서 개발

격리했던 7건의 원인을 근본까지 추적했습니다. 기존 추출은 HWP5의 `BodyText/SectionN` 스트림을 레코드 타입 구분 없이 통째로 UTF-16으로 디코딩하고 있었습니다. 그 결과 표·도형 레코드의 바이너리가 "비어있지 않은 문자열"로 통과되어 노이즈로 쌓였습니다.

EIP3.0 문서 사례: raw 디코딩 시 124,631자, zlib 압축해제 후 디코딩해도 383,168자가 나오지만 둘 다 표/도형 레코드 바이너리가 텍스트 사이에 낀 노이즈였습니다. v2에서 시도한 `PrvText` (미리보기 캐시) 방식은 오히려 234자 → 52자로 더 줄었습니다.

해결책은 레코드 구조를 존중하는 파싱입니다. HWP5의 `BodyText`는 4바이트 레코드 헤더(`tag_id 10bit + level 10bit + size 12bit`)로 시작하는 레코드들의 연속이고, 그중 `HWP_TAG_PARA_TEXT` (`tag_id=67`) 레코드만 실제 문단 텍스트(UTF-16LE)입니다. 새 파서는 다음을 수행합니다.

- 레코드 헤더 4바이트를 읽어 `tag_id/size`를 구하고, `size == 0xFFFF` 면 확장 size(4바이트)를 추가로 읽습니다.
- `tag_id=67`인 레코드만 문단 텍스트로 디코딩하고, 나머지(표·도형·글자모양 등)는 size만큼 건너뛴니다.
- 문단 텍스트 레코드 내부에서도 코드값 32 미만의 제어문자(탭·개행 제외)는 인라인 컨트롤을 가리키며 뒤에 WCHAR 7개(14바이트)의 부가정보가 따라오므로 함께 건너뛴니다.
- 일반 텍스트는 한 글자씩이 아니라 바이트 버퍼에 모아 한 번에 `utf-16-le` 로 디코딩합니다(한 글자씩 디코딩하면 surrogate pair가 반쪽씩 잘려 오류 발생).

PDF는 별개 문제였습니다. 서울시립대 문서는 149페이지짜리인데 기존 코드가 `reader.pages[0]` 처럼 첫 페이지만 접근해 220자만 추출했습니다. 전체 페이지를 순회하도록 수정했습니다.

복원 결과:

문서	기존(vendor)	재추출 후	배율
EIP3.0 고압가스 안전관리	234자	105,255자	450배
스포츠윤리센터 LMS	298자	80,361자	270배
서울시립대 학업성취도(pdf)	220자	111,242자	506배
구미아시아육상경기	186자	50,002자	269배
건설기술 특허·실용신안	89자	60,356자	678배
JST 공유대학 xAPI LRS	130자	76,123자	586배
사업장 사회보험료 보완 개발	401자	67,455자	168배

복원 후 전체 100건에서 500자 미만 행은 0건, 최솟값 533자가 되었습니다. 노이즈 검사(제어문자··lone surrogate·mojibake)도 전부 0건으로 확인되었습니다.

이 단계에서 부수적으로 **메타데이터 결측**과 **"마감=개찰-1시간" 구조적 오류**의 단서를 처음 포착했습니다. 5개 핵심 컬럼 중 하나라도 결측인 행이 35건 이었고, 한영대·수도국산달동네박물관 문서에서 본문에 적힌 마감 시각과 CSV의 마감일이 정확히 1시간씩 차이 나는 패턴을 발견했습니다.

3.3 골든셋 최초 생성과 메타데이터 구조 오류 수정

`eval_questions_v3.csv` (200문항)를 만들면서 메타데이터를 나라장터와 대조하는 과정에서 두 가지 구조적 문제를 확정했습니다.

① **"마감일"이 사실은 "개찰일"이었던 오류**: 기존 CSV의 **입찰 참여 마감일**이 실제 입찰서 제출 마감일이 아니라 그보다 1시간 늦은 개찰시각으로 채워져 있었습니다. KORAIL 시스템에서 입찰마감과 개찰이 별도 필드로 구분된다는 것을 확인해 메커니즘까지 규명했습니다(마감 후 1시간 뒤 개찰하는 행정 절차).

사업명	기존(개찰시각)	정정(실제 마감)
트랙운영 학사정보시스템 고도화(한영대학)	17:00	16:00
수도국산달동네박물관 전시해설 시스템	18:00	17:00
예약발매시스템 개량 ISMP(한국철도공사)	06-11 00:00	06-12 10:00 (날짜 자체 오류)

② **중복 문서 2건**: 같은 RFP가 서로 다른 발주기관 라벨로 두 행에 중복 입력되어 있었습니다. 한쪽은 라벨은 맞지만 텍스트가 잘려 있고, 다른 쪽은 텍스트는 완전하지만 라벨이 틀린 패턴이었습니다.

사업명	유지	삭제
통합정보시스템 고도화 용역	한국한의학연구원	국가과학기술지식정보서비스
의료기기산업 종합정보시스템 기능개선	한국보건산업진흥원	BioIN "(2차)"

이로써 행 수가 **100 → 98행**이 되었고, 결측은 나라장터·기관별 전자조달시스템(K-water, KORAIL, KOICA 등) 직접 조회로 32건 → 17건으로 축소했습니다.

3.4 91건(93%) 전량 재추출

결측·이상치를 점검하던 중 텍스트가 500~1,500자에 불과한 문서 21건이 추가로 발견됐습니다. 열어보면 "주주평등원칙(상", "[별첨 3] 소프트웨어"처럼 단어·목차 항목 중간에서 끊겨 있었습니다. 원본 hwp 파일 크기((사)벤처기업협회 4.2MB)와 추출 길이(503자)를 비교하면 명백한 추출 실패였습니다.

원인은 파서의 부분 적용이었습니다. 레코드 기반 파서가 실제로는 7건에만 적용되고, 나머지 91건(hwp 88 + pdf 3)에는 반영되지 않은 채 방치되어 있었습니다. `document_loader.py`에 같은 방식의 수정이 이미 작성돼 있었으나 `data_list_버전3.csv`에는 반영되지 않은 상태였습니다.

표본 5건으로 검증하니 새 파서로 평균 32배 더 많은 본문이 나왔고, 글자 단위 prefix 대조로 "새 텍스트가 기존 텍스트의 이어지는 연속"임을 확인했습니다(요약이 아니라 끊긴 지점부터 다시 읽은 것). `reextract_all_text_v3.py`로 전체 98행에 적용했습니다(고려대 297페이지 PDF가 기존엔 1페이지만 추출되던 것도 이때 수정).

항목	재추출 전	재추출 후
텍스트 최솟값	89자	42,769자
텍스트 평균	9,115자	72,802자
결측 건수	17건	0건

결측 17건은 나라장터/기관별 전자조달시스템 직접 조회로 전부 보완했습니다. 이 과정에서 "상시접수 → 공고게시일로 대체" 규칙을 확립했습니다(입찰서 접수가 특정 시작일 없이 상시 진행되는 8건에 공고게시일을 시작일로 사용). 사업금액 이상치(0원/1원)도 정리해, 을지대·보험개발원처럼 "의도적 비공개"인 2건만 남기고 모두 해소했습니다.

이 단계에서 eval_questions_v4.csv (344문항)를 생성했으며, LLM 기반 카테고리(일정/자격/제출서류) 25건 샘플 감사로 hallucination 0건을 확인했습니다.

4. 청킹 파이프라인 설계와 개선

4.1 청킹 파이프라인 구축과 두 가지 숨은 버그

`data_list_버전4.csv` (98건)를 16,218개 청크로 분할하는 과정에서 두 가지 버그를 발견하고 같은 단계에서 수정했습니다.

버그 ① — `engine: langchain` 설정의 조용한 fallback config는 `rag.engine: langchain`, `rag.splitter.type: recursive_character`로 되어 있어 LangChain의 문장 경계 인식 분할기가 쓰일 것처럼 보였습니다. 그러나 `langchain-text-splitters`가 미설치라, 조건이 맞으면 (`embedding=local`, `answerer=local`, `vector_store=memory` — 정확히 이 프로젝트 config) `src/rag/chunker.py`의 단순 글자수 분할로 **조용히 fallback하고 있었습니다. 증거로 청크 경계를 열어보면 숫자 중간에서 끊김**

```
20241001798_chunk_0001 ... 끝부분: "... □ 사업예산 : 130," ← 숫자 중간에서 끊김
```

실제로 동작한 로직은 문장 경계를 전혀 고려하지 않고 글자 수로만 자르는 방식이었습니다.

```
def _split_text(text, chunk_size, overlap):
    clean_text = " ".join(text.split()) # 공백/줄바꿈을 한 칸으로 압축
    start = 0
    while start < len(clean_text):
        end = min(start + chunk_size, len(clean_text))
        chunks.append(clean_text[start:end])
        if end == len(clean_text):
            break
        start = end - overlap
```

재현성 이슈: 누군가의 환경에 `langchain-text-splitters`가 설치돼 있으면 같은 config·같은 문서로도 실제 분할 결과(청크 경계·개수)가 달라질 수 있습니다. 결과를 비교할 때는 `pip list | grep langchain`으로 환경을 먼저 맞춰야 합니다.

버그 ② — 메타데이터가 검색 대상 밖에 있었음 `_parse_csv_document()`가 `사업 금액 / 발주 기관`을 `meta_*` 필드로만 저장했는데, 청킹 로직은 `document["text"]`만 자르기 때문에 검증된 메타데이터 값이 **청크 텍스트에 한 번도 들어간 적이 없었습니다**. 그 결과 예산 질문 39건이 어떤 청크에서도 매칭되지 않았습니다. 추가로 39건 중 35건은 그 금액이 RFP 본문 어디에도 등장하지 않았습니다(포털 메타데이터로만 존재).

임시 조치 — 첫 청크에만 프리앰블 삽입

```
"text": _normalize_text(f"{_build_meta_preamble(entry, title)}{text}"),
# 예: "사업명: 한영대학교 ... 고도화 | 발주기관: 한영대학 | 사업금액: 130,000,000원 | 공고차수: 0.0 <본문...>"
```

재ingest 후 매칭이 발주기관 79/98 → 98/98, 예산 57/96 → 96/96으로 개선됐습니다. **다만 이 프리앰블은 문서의 첫 번째 청크(`chunk_0001`)에만 붙는 방식이었고, 이 한계가 이후 청킹 개편 단계에서 다시 문제로 드러납니다.**

4.2 성능 병목 규명과 청킹 전면 개편

앞 단계에서 만든 데이터로 실험한 결과(`rag_golden_v6_final`, 378문항)를 카테고리별로 분석하니 처참했습니다.

카테고리	문항 수	retrieval hit rate
예산	96건	37.4%
발주기관	97건	29.9%
일정(사업기간)	83건	8.4%
자격	48건	10.4%
제출서류	51건	0.0%

메타데이터 기반인 예산·발주기관조차 30% 안팎이었고, 제출서류는 전 문항 실패였습니다. 구조적 원인 두 가지를 규명했습니다.

원인 ① — 메타 프리앰블이 `chunk_0001`에만 존재 문서 메타(사업명/발주기관/사업금액)가 첫 청크 앞에만 붙어 있어, retriever가 `chunk_0002`이후를 반환하면 예산·발주기관 정보를 찾을 방법이 없었습니다. 인접 청크 오검색(오버랩 80자로 인한 임베딩 유사도 혼동)도 잦았습니다.

원인 ② — `chunk_0001`이 목차 노이즈로 오염 HWP 문서 앞부분은 대부분 목차인데, 목차 항목은 탭 문자와 HWP 페이지번호 특수문자로 구성되어 임

베딩이 노이즈에 희석됩니다. 첫 500자 내 탭 10개 이상(목차 의심)인 문서가 98개 중 88개였고, 나머지 10개만 바로 본문 시작이었습니다.

개선책 — build_chunks_v2.py 신규 작성(3가지 변경)

- 1. **목차 제거**: 처음 시도한 "목차 헤더~본문 시작 구간 통째 삭제" 방식이 "이러닝시스템 운영 용역" 문서의 사업기간 본문까지 삭제하는 버그를 일으켰습니다. 이를 폐기하고 다음 두 조건을 만족하는 **줄만 개별 제거**하는 방식으로 바꿨습니다. - 목\s+차 패턴에 매칭되고 길이 < 20자인 줄 (목차 헤더) - HWP 페이지번호 특수문자 포함 + 탭 있음 + 길이 < 80자인 줄 (목차 항목)
- 2. **전 청크에 메타 헤더 삽입**: [사업명: XX | 발주기관: YY | 사업금액: ZZ | 공고차수: N] 을 모든 청크 앞에 삽입. 어느 청크가 검색되든 발주기관·사업금액 정보를 포함하게 됩니다.
- 3. **오버랩 80 → 40자 축소**: 인접 청크 간 공유 텍스트를 줄여 임베딩 유사도 혼동을 완화.

항목	버전4 기반 기준	버전5(신규)
총 청크 수	16,222개	14,717개
목차 제거 문서	0개	88개/98개
메타 헤더 위치	chunk_0001만	전 청크
오버랩	80자	40자

청크 수가 1,505개 감소한 것은 목차 텍스트 제거에 따른 정상 감소입니다. 청크 평균 길이는 프리앰블(~90자)이 추가돼 약 588자가 되었습니다(본문은 여전히 약 500자). 빈 청크·중복 chunk_id·메타 헤더 누락 청크는 모두 0건으로 검증됐습니다.

4.3 최종 실험용 청킹 설정 (버전5)

- 소스: data_list_버전5.csv (98개 문서)
- 분할: 글자수 기반, chunk_size 500
- 오버랩: 40자
- 모든 청크 앞에 [사업명 | 발주기관 | 사업금액 | 공고차수] 메타 헤더
- 목차 줄 제거 적용 (88/98 문서)
- 총 14,717 청크
- 이 청크 기준으로 eval_questions 의 expected_chunk_ids 재매핑 (378/378 = 100%, 미매칭 0)

팀 공유 미결 이슈: 서비스 corpus는 오버랩 80(약 16,700 청크), 실험 corpus는 오버랩 40(14,717 청크)로 설정이 갈려 있습니다. 어느 쪽을 정본으로 할지 팀 결정이 필요합니다.

5. 평가용 골든 데이터셋 구축

5.1 골든셋의 역할과 구조

RAG 성능을 측정하려면 "이 질문에 대한 정답은 이것이고, 그 정답은 이 청크에 있다"는 기준 데이터가 필요합니다. 이것이 골든 데이터셋(golden dataset)입니다. 최종 미니셋 `eval_questions_v12_mini50.csv` 의 컬럼 구조는 다음과 같습니다.

컬럼	의미
<code>question</code>	평가 질문 (사업명 + 카테고리형 질문)
<code>expected_answer</code>	정답 (문서 원문에서 발췌한 verbatim 정답)
<code>expected_chunk_ids</code>	정답이 들어있는 청크 ID 목록
<code>category</code>	질문 유형 (발주기관/예산/일정/자격/제출서류)
<code>source_doc_id</code>	출처 공고 문서 번호
<code>note</code>	검증 이력 메모
<code>chunk_text</code> / <code>answer_in_chunk_text</code>	정답이 청크 원문에 실제로 있는지 검증 결과

5.2 카테고리별 생성 전략

카테고리마다 정답의 출처가 다르므로 생성 방식도 다릅니다.

- **발주기관·예산**: CSV의 검증된 컬럼값을 그대로 정답으로 사용(LLM 미사용). hallucination 위험이 없습니다.
- **일정·자격·제출서류**: 정답이 본문에 문장으로 묻혀 있어 GPT-4o-mini(temperature=0)로 의미 기반 추출. "문서에 없으면 '정보없음'이라고만 답하라"는 프롬프트 제약을 걸고, 그렇게 나온 답변은 최종 데이터셋에서 제외했습니다.

자격·제출서류 질문 수가 상대적으로 적었던 이유가 있습니다. 제출서류는 문서 앞쪽 목차의 "IV. 제출서류" 항목을 LLM이 실제 목록으로 오인하거나, 실제 목록이 문서 맨 뒤 별지 서식에 있어 LLM 입력(앞 3,000자) 범위 밖인 경우가 많았습니다. 자격은 애초에 전용 컬럼이 없고 본문 뒤쪽에 있는 경우가 많아 마찬가지로 문제를 겪었습니다.

5.3 chunk_id 매핑과 직접 대조

`fill_expected_chunk_ids.py` 로 각 질문의 정답이 어느 청크에서 나왔는지 채웠습니다. exact 매칭(정답 문자열이 청크에 그대로 존재) → fuzzy fallback(키워드 중첩 최대 청크) 순으로 동작합니다. 최종적으로 344/344(100%)를 채웠고, exact로 안 채워진 48건(자격 22, 제출서류 23, 일정 3)은 **전부 직접 원문을 열어 대조**했습니다.

이 과정에서 중요한 구분을 발견했습니다. **"장(章) 목차"와 "서식(form) 목차"는 다릅니다.** "I. 사업개요 II. 구축방안..." 같은 장 목차는 제목만 있고 내용이 없어 잘못된 배정이지만, "[서식1] 일반현황, [서식2] 자본금 현황..." 같은 서식 목차는 서식 이름 자체가 제출서류 답변과 같아 그대로 정답이 됩니다. 자격 카테고리에서 9건, 제출서류에서 3건의 목차/다른 섹션 오배정을 교정했습니다.

또한 청크 자체의 무결성도 점검했습니다. overlap을 "80자 정확히"로만 보면 16,120쌍 중 6,742쌍이 어긋나 보였지만, 직접 열어보니 `_split_text()` 가 각 청크 끝에 `.strip()` 을 적용해 공백이 걸리면 78~79자로 1~2자 줄어드는 것뿐이었습니다(실제 겹침: 80자 9,378쌍, 79자 5,811쌍, 78자 931쌍 — 전부 70자 이상). 내용이 실제로 빠진 경우는 0건이었습니다.

5.4 verbatim 검증과 v6 확정

`eval_questions_v4_with_chunk_text.csv` (344행)를 점검하니 `answer_in_chunk_text=False` (정답 문구가 청크 원문에 그대로 없음) 항목이 77개(22%)였습니다. 원인은 정답 표현과 청크 원문의 띄어쓰기·줄바꿈 불일치가 대부분이었습니다.

- GPT-4o-mini로 청크에서 정답을 다시 verbatim 추출해 40건 자동 수정(77→37)
- GPT-4o+정규화 매칭으로 3건 추가 수정(37→34)
- 최종 34건은 "청크에 실제 정답 없음"으로 판정해 삭제

삭제 후 재생성과 카테고리 균형을 거쳐 최종 산출물을 확정했습니다.

단계	처리	결과
False 34건 삭제	344 → 310개(100% True)	검증 불가 항목 폐기
삭제분 대체 신규 생성	+22개	삭제 문서에서 재생성
자격·제출서류 목표치 증량	+46개	자격 50 / 제출서류 50
이상 답변 3개 재생성	정제 완료	—

최종 `eval_questions_v6_final.csv` 는 **378개**(발주기관 98 / 예산 96 / 일정 84 / 자격 50 / 제출서류 50), `answer_in_chunk_text = True` 378/378(100%)입니다.

5.5 버전5 청크 기준 재매핑

버전5 청크가 새로 만들어졌으므로 v6_final의 `expected_chunk_ids` 를 재매핑했다: exact 334 + fuzzy 44 = **378/378(100%), 미매칭 0**. 이 산출물이 `eval_questions_v7_final.csv` 입니다. 처음 실행했을 때 미매칭 2건이 있었는데, 둘 다 목차 제거 로직의 "범위 삭제" 초기 버전 버그가 원인이었고 "줄 단위 필터"로 수정 후 해소했습니다.

5.6 라벨 오배정과 콘텐츠 결합 정밀 교정

라벨 교정 단계는 "청크 라벨"을, 콘텐츠 검증 단계는 "질문·답변 콘텐츠 자체"의 결합을 파고들었습니다. 이 단계에서 반복적으로 나타난 버그 유형을 정리하면 다음과 같습니다.

- ① **fuzzy 오배정**: `expected_answer` 가 원문에 verbatim으로 없으면(주로 LLM이 paraphrase한 경우) 키워드 겹침만으로 청크를 확정해 무관한 청크를 정답으로 가리켰습니다. 원문 대조로 42/56건 재검증·수정했습니다.
- ② **HWP 잔재 문자 + 정규식 인코딩 버그**: `expected_answer` 에 HWP 페이지번호 한자(珎, 第 등)가 박혀 있었는데, v5는 그 문자가 제거된 정제 텍스트라 검증 스크립트가 리터럴 비교에서 "못 찾음"으로 오판정했습니다. 게다가 정규식에 한자를 리터럴로 입력하니 겹보기엔 같은 글자라도 코드포인트가 달라(豈 U+8C48 vs U+F900) 한글 음절까지 삭제되는 추가 버그가 발생했습니다. 모든 정규식을 `\uXXXX` 이스케이프로 통일해 근본 차단했습니다.
- ③ **괄호 안 쉼표 오분리**: "실적증명원(공사, 물품, 용역)" 이 `answer.split(",")` 때문에 3조각으로 잘렸습니다. 괄호 깊이를 추적하는 `_split_respecting_parens()` 와, 항목의 머리/꼬리가 어느 청크에 있는지 찾아 그 구간을 묶는 `find_span()` 방식으로 교체했습니다.
- ④ **콘텐츠 자체 결합**: 답변이 질문과 다른 주제(제재조항을 자격요건으로 답함), 목차 제목 한 줄뿐, 사업명 자체가 틀린 질문 제목 등 원본 콘텐츠 결합 37건을 찾았습니다. HWP 잔재만 있는 21건은 문자 제거, 나머지 16건은 원문에서 실제 섹션을 찾아 재작성했습니다.

일정 카테고리 84건은 사업명 오류 우려로 전수 원문 대조했으나 84건 전부 정상으로 확인했습니다. 최종 결과물은 `eval_questions_v11_final.csv` (371건: 358건 완전 검증 + 13건 청크만 검증)입니다.

5.7 실제 실험 결과로 발견한 질문 제목 문제

이 단계에서는 처음으로 실제 RAG 파이프라인 실험 결과(`rag-baseline-v11mini-bm25` 의 bad_retrievals 10건)를 근거로 새 결합을 찾았습니다.

- **패턴 A(6/10)**: 문서는 찾았지만 top_k=10 안에 정답 청크(문서 뒤쪽 별지서식에 몰림)가 없음 → 리트리버 설정 문제
- **패턴 B(4/10)**: 질문 제목이 "SW산출물 사업의", "하도급 관리방안 사업의"처럼 **실제 사업명이 아니라 일반명사형 카테고리 이름** → 검색기가 어느 문서인지 특정 못 함. config로 해결 불가능한 데이터셋 문제

질문 371건을 전수 스캔해 26건에서 제목이 실제 사업명과 무관함을 확인하고, `question` 컬럼의 사업명 부분만 실제 사업명으로 치환했습니다(내용·청크ID 불변). 또한 예산 카테고리의 `expected_chunk_ids` 라벨을 발주기관과 동일하게 문서 전체 청크로 통일(6건 확장)했습니다. 최종 산출물은 `eval_questions_v12_final.csv` (370건)입니다.

5.8 골든셋 예시

- ① **발주기관 유형** — 정답이 명확한 단답, 정답 청크는 문서 전체

질문: 건설통합시스템(CMS) 고도화 사업의 발주 기관은 어디입니까?
 정답: 한국수자원공사
 카테고리: 발주기관
 정답 청크: 문서 전체 134개 청크

- ② **예산 유형** — 부가 문구가 붙는 경우, 정답 청크는 문서 전체

질문: 광주정책연구아카이브(GPA) 시스템 개발 사업의 예산은 얼마입니까?

정답: 금43,000,000원 이내 (부가가치세 포함)

카테고리: 예산

정답 청크: 문서 전체 146개 청크

③ 제출서류 유형 — 원문에 없어 "정보 없음"을 정직하게 답하는 경우, 정답 청크는 단일

질문: 연차별 자율 품질관리 시스템 기능개선 사업의 제출 서류는 무엇입니까?

정답: 본 문서에는 제출서류 세부 목록이 명시되어 있지 않음.

「사. 제안서 등록 및 구비서류」 항목에서 "(제출서류) 「입찰공고문」 참조"라고

안내하고 있어, 제출서류 목록은 별도 공고문에서 확인해야 함

카테고리: 제출서류

정답 청크: 20240429895_chunk_0086 (단일 청크)

①·②는 정답 청크가 "문서 전체"인데, 발주기관·예산은 메타 헤더 덕분에 문서 어느 청크가 검색돼도 정보가 들어 있어 문서의 모든 청크를 정답으로 인정하는 방식입니다. ③은 특정 조항에만 답이 있어 그 청크만 지정하며, "정보 없음"도 정직한 정답으로 다뤄 데이터셋 품질을 관리한 사례입니다.

6. RAG 실험 및 하이퍼파라미터 튜닝

실험 목표: retrieval_hit $\geq$ 75% / citation $\geq$ 65% / judge_correct $\geq$ 80% / not_found $\leq$ 15%

6.1 임베딩 / 답변 모델 변경 이력

임베딩은 Ollama(로컬) → OpenAI로 딱 한 번 교체했고, 이후 계속 OpenAI로 유지했습니다. 답변 생성 모델은 처음부터 끝까지 gpt-5-mini였습니다.

그룹	임베딩 모델	답변 모델	교체 원인	결과
baseline	Ollama nomic-embed-text	gpt-5-mini	베이스라인	retrieval_hit 29.5% — 한국어 RFP 의미 파악 부족
emb-openai 이후	OpenAI text-embedding-3-small	gpt-5-mini	로컬 임베딩 한계	retrieval_hit 78.5%로 대폭 개선, 이후 기본값 고정

테스트만 한 모델: exaone3.5:7.8b(로컬 한국어 특화 LLM)로 답변 생성기를 교체 시도했으나 성능이 떨어져 gpt-5-mini를 유지했습니다.

핵심: 최종까지 쓰인 조합은 OpenAI text-embedding-3-small + gpt-5-mini 하나뿐이며, top_k · reranker · BM25 같은 retrieval 단계 파라미터만 계속 조정했습니다.

6.2 평가 체계의 함정 두 가지

성능을 정확히 측정하기까지 두 개의 측정 버그를 건어내야 했습니다.

① **not_found 측정 버그**: 초기에는 "검색 결과가 하나라도 있으면 무조건 답변함"으로 기록해, 모델이 "확인하지 못했습니다"라고 답해도 not_found로 잡히지 않았습니다. 판정 기준을 "답변 텍스트에 fallback 문구가 있는지"로 수정하니 not_found_rate가 처음으로 정확히(42%) 측정됐습니다. judge_correct는 51.5→51%로 거의 그대로였다 — 정답률이 떨어진 게 아니라 측정 오류가 드러난 것입니다.

② **expected_chunk_ids 과대매핑**: v4 데이터로 교체하니 retrieval_hit_rate가 95.6%로 치솟았으나 과대평가였습니다. expected_chunk_ids를 "그 문서의 모든 chunk"로 매핑해서, 같은 문서의 아무 청크나 검색되면 무조건 hit으로 잡히는 구조적 문제였습니다. 정밀 매핑한 v6_mini50(50문항)으로 바꾸니 retrieval_hit가 30%로 떨어졌고 이것이 진짜 값이었습니다. not_found 88건을 분석하니 94.3%(83건)가 모델 회피가 아니라 retrieval 정밀도 문제(정답 없는 청크를 가져와 모델이 "확인 못 함"이라 답함)였습니다.

또한 채점 방식을 **string matching**에서 **LLM judge**로 전환했습니다. 기존에는 "200,000,000원"을 "2억원"으로 답하면 오답 처리됐는데, LLM이 의미 기준으로 채점하면서 정답률을 정확히 측정할 수 있게 됐습니다.

6.3 Retrieval 고도화

단계	핵심 변경	retrieval_hit	judge_correct	비고
베이스라인	Ollama nomic	29.5%	–	한국어 의미 파악 부족
임베딩 교체	OpenAI text-embedding-3-small	78.5%	–	retrieval 대폭 개선
llm_judge 도입	의미 기반 채점	78.5%	51.5%	정답률 첫 측정
정밀 매핑(v6mini)	과대평가 제거	30%	72%	실측값 확인
reranker 도입	Cross-Encoder(BAAI/bge-reranker-v2-m3) 재정렬	34%	78%(+6%p)	hit@1 8%→28%
BM25+RRF	RRF 결합 + fetch_k 보정	86%	76%	retrieval 목표 달성
top_k=15+reranker	최종 설정	94~96%	82%	전 지표 목표 달성

reranker는 초기 검색 결과를 관련도 순으로 재정렬해 judge_correct를 크게 끌어올렸고 hit@1을 3배 이상 개선했습니다.

BM25+RRF는 두 단계를 거쳤습니다. 첫 시도는 BM25 결과를 semantic 결과 뒤에 단순 append했는데, semantic이 이미 fetch_k를 채우면 BM25 후보가 전부 잘려나가는 버그가 있었습니다. semantic과 BM25는 점수 척도가 달라(코사인 0~1 vs BM25 10~25) 직접 비교가 불가하므로, 순위만 보고 $1/(60+\text{rank})$ 로 점수를 매겨 합산하는 **RRF(Reciprocal Rank Fusion)** 방식으로 수정하고, hybrid일 때 $\text{fetch_k}=\text{top_k} \times 3$ 으로 늘려 BM25 자리를 확보했습니다. 결과: retrieval_hit 80→86%, hit@1 48→58%.

6.4 하이퍼파라미터 전후 비교(요약)

파라미터	변경	before → after	판단
embedding_model	Ollama → OpenAI	retrieval_hit 29.5% → 78.5%	채택
retriever.top_k	10 → 15	judge 70% → 76%	채택
answerer.temperature	0.2 → 0.0	judge 66.7% → 71.4%	채택(0.0 고정)
reranker.enabled	false → true	judge 76% → 82%, failure 14%→6%	채택
retriever.hybrid_bm25	false → true(RRF)	retrieval 80% → 86%	채택
answerer.prompt	기본 → 4원칙	judge 74% → 68%	기각(역효과)
동의어 사전 확장	—	judge 76% → 72%	기각(노이즈)
BM25 tokenizer	split() → kiwi	judge 76% → 72%, hit@1 하락	기각
answerer	gpt-5-mini → EXAONE 3.5	judge 76% → 68%	기각
데이터 버전	v11 → v12	retrieval_hit 94% → 96%	채택

실패한 시도들의 교훈: prompt 4원칙 추가는 모델이 원칙 문구를 답변에 그대로 노출해 judge에 불리하게 작용했습니다. kiwi 형태소 분석은 "사업기간"→"사업"+"기간"으로 쪼개 "용역기간"과 매칭시키려는 의도였으나 hit@1이 오히려 하락했습니다(v12에도 적용하고 싶었으나 토큰 부족으로 미진행). EXAONE은 이 테스트에서 gpt-5-mini보다 부적합했습니다.

6.5 최종 실험 결과 (v11/v12, top_k + reranker)

실험	top_k	reranker	retrieval_hit	hit@1	citation	judge_correct	failure
v11mini top10	10	off	80%	54%	80%	70%	20%
v11mini top10+reranker	10	on	92%	76%	92%	74%	8%
v11mini top15	15	off	86%	54%	84%	76%	14%
v11mini top15+reranker	15	on	94%	74%	90%	82%	6%
v12mini top15	15	off	86%	60%	84%	70%	14%
v12mini top15+reranker	15	on	96%	80%	92%	80%	4%

top_k 10→15만으로 judge 70→76% 개선, reranker 추가로 failure 20→6%, judge 76→82%로 가장 큰 폭 개선. v12는 retrieval 지표가 더 높지만(96%) judge는 v11보다 약간 낮았습니다(80% vs 82%).

7. 하이퍼파라미터 전후 답변 비교

실제 RAG UI에서 같은 질문에 파라미터만 바꿔 뽑은 답변 비교입니다. 지표 개선이 실제 답변 품질에서 어떻게 드러나는지 보여줍니다.

7.1 reranker: off → on

질문: 2024년 항해용 간행물 품질관리 업무보조 시스템 구축 사업의 참가 자격 요건은?

구분	답변
Before (off)	학술연구용역·해도제작업·컴퓨터관련서비스업 등록, 중소기업확인서 — 4개 등록요건만
After (on)	위 4개 + 직접생산확인증명서, 부정당업자 참가제한, 공동수급 요건까지 취합
정답	4개 등록요건 + 중소기업확인서 + 직접생산확인증명서 + 부정당업자 제한 미해당

reranker가 관련도 높은 청크를 상위로 재정렬하면서 누락됐던 근거가 추가로 포함됐습니다.

7.2 hybrid BM25: off → on

질문: 하도급 관리방안 사업의 제출 서류는?

구분	답변
Before	하도급 사업수행 계획서, 물품 공급확약서, [별지양식8] 증빙자료만
After	위 항목 + 소프트웨어사업 하도급계획서(별지 서식 제2호·제3호) 추가
정답	『소프트웨어사업 하도급 승인 및 관리 지침』 별지 서식 제2호·제3호 하도급계획서

벡터 검색이 놓친 키워드(서식 제2호/제3호) 매칭 문서를 BM25가 보완했습니다.

7.3 BM25 tokenizer: split() → kiwi

질문: 세종테크노파크 인사정보 전산시스템 구축 용역 사업의 사업기간은?

구분	답변
Before (split)	"계약 체결일로부터 약 60일 이내" + 불필요한 부연("오픈은 3개월 이내") 병기
After (kiwi)	"용역기간은 계약 체결일로부터 약 60일 이내입니다." (간결)
정답	계약 체결일로부터 약 60일 이내

이 개별 사례에서는 답변이 간결해졌지만, 전체 지표(hit@1)에서는 역효과가 나 최종 채택하지 않았습니다. 개별 예시와 집계 지표가 어긋날 수 있음을 보여주는 사례입니다.

7.4 eval_question 데이터 버전: v11 → v12

질문: 학사시스템 기능개선 사업의 제출 서류는?

구분	답변
Before (v11)	입찰참가서류·산출물을 나열하나 서식·번호 체계 없이 답함
After (v12)	서식1(입찰참가신청서) 등 개별 서식 번호를 명시, 정답 형식에 근접
정답	서식1~11 (입찰참가신청서, 가격제안서, 주요사업실적 ... 청렴계약 이행각서)

v12의 문서 파싱/청킹 개선으로 서식 번호 정보가 청크에 더 잘 보존되어 답변에 반영됐습니다.

8. Vector DB 선정

결정: ChromaDB (우선 적용), 규모 확장 시 PostgreSQL + pgvector

8.1 선정 기준

기준	설명
배포 용이성	Python 3.11 + conda 환경에서 추가 설치 없이 사용 가능
초기 규모	문서 100건, chunk 약 2,000~5,000건에서 충분한 성능
LangChain 통합	기존 LangChain 엔진과 자연스럽게 연결
메타데이터 필터	document_id, section, page 기준 필터링
확장 가능성	추후 PostgreSQL+pgvector 등으로 마이그레이션 가능한 구조

8.2 후보 비교

	ChromaDB	FAISS	LanceDB	Qdrant
설치	<code>pip install chromadb</code>	<code>conda install faiss-cpu</code>	<code>pip install lancedb</code>	Docker 권장
의존성	이미 requirements에 포함	추가 필요	추가 필요	추가 필요
속도(소규모)	빠름(5K건 수ms)	매우 빠름	빠름	빠름
속도(대규모)	100K+ 저하	100만건 실시간	100만건 실시간	수백만건 실시간
메타데이터 필터	O(when)	직접 구현	O(컬럼 기반)	O(payload index)
Full-Text Search	X	X	X	O
영속성	자동(SQLite)	수동	자동(Lance)	자동
LangChain 통합	O(공식)	O(community)	X	O(community)
적용 난이도	하	중	중	상

8.3 ChromaDB 채택 사유

- 의존성 추가 제로** — requirements.txt 에 langchain-chroma==1.0.0 이 이미 포함되어 추가 설치 없이 사용 가능
- 구현 난이도 최저** — LangChain 공식 통합으로 Chroma.as_retriever() 한 줄이면 검색기 완성. FAISS처럼 index 저장/로드를 직접 구현할 필요 없음
- 현재 규모에 적합** — 문서 100건, chunk 5,000건 수준에서 수ms 응답
- 영속성 기본 제공** — SQLite 기반으로 서버 재시작 후에도 유지

8.4 한계 및 대응

한계	대응
100K+ chunk에서 속도 저하	현재 규모 문제없음, 규모 확대 시 PostgreSQL+pgvector 검토
Full-Text Search 미지원	SQLite FTS5 또는 기존 keyword retriever와 병행
단일 머신 전용(분산 안 됨)	현재 VM 단일 서버로 충분, 분산 필요 시 Qdrant 검토

8.5 적용 범위

[변경 전]	[변경 후]
embeddings.jsonl 파일	ChromaDB Collection
↓	↓
ChatbotRunner.load_document_context()	chatbot.connect_collection(run_id)
↓	↓
self.embeddings (list in memory)	collection.query(query_embedding)

- `adapters.py` → `ChromaRetrieverAdapter` 신규 추가
 - `chatbot.py` → `load_document_context()` 를 Chroma connection으로 대체
 - `rag_service.py` → `_chatbot_cache` 를 Chroma client 관리로 변경
-

9. 프론트엔드(웹 UI) 구현

RAG 백엔드와 별개로, 사용자가 실제로 마주하는 웹 화면(Streamlit)을 구축·개편한 작업입니다. 이 단계는 데이터 연동을 Mock 함수로 대체해두고, 실제 RAG 모델 완성 시 해당 함수만 교체하면 되도록 설계했습니다.

9.1 프론트엔드 개요

항목	내용
서비스명	IT'S MINE (구 BidAI)
성격	RAG 기반 공공 RFP 문서 분석 웹 서비스(데모)
프론트엔드	Streamlit 멀티페이지 (st.navigation)
진입 파일	app.py
실행 환경	Linux / Python 3.11.15 / Streamlit 1.58.0
구조	화면 코드는 views/, 공통 코드는 utils/ 폴더로 분리

앱은 파일 업로드가 아니라 **내부 corpus로 미리 인덱싱해둔 RFP 문서 목록**을 기반으로 동작합니다. 사용자가 문서를 검색·선택하면 요약·요구사항 추출·채팅 질의를 수행하는 흐름이며, RAG 미연결 시 `utils/mock_data.py` 의 mock 데이터로 화면을 미리볼 수 있습니다.

9.2 확정된 요구사항

작업은 대화를 진행하며 순차적으로 확정되었으며, 최종 요구사항은 다음과 같습니다.

- 상단바(topbar) 구성** — **IT'S MINE** (서비스명, 크게) · **서비스 소개** · **정부제안서 검색** · **요금제** 배치. 레이아웃은 A안(왼쪽 끝 브랜드 + 오른쪽 끝 메뉴 3개 균등 배치).
- 홈 연결** — **IT'S MINE** 클릭 시 홈 화면으로 이동.
- 서비스 소개 페이지 신규 생성** — 홈이 아닌 독립 페이지로 분리(`views/about.py`).
- 정부제안서 검색 페이지** — 외부 사이트 링크 모음. 4열 × 2행 = 8칸 아이콘 그리드, 각 칸에 아이콘 + 사이트명 + 주소, 클릭 시 새 탭.
- 상단바 메뉴 hover 효과** — 마우스 오버 시 시각적 반응(밑줄 제외, 색·배경 변화). 상단바 메뉴에만 적용.
- 사이드바 완전 삭제** — 자동 생성되는 좌측 사이드바 메뉴 제거. 페이지 이동은 상단바로만.
- 브랜드 표기 통일** — 코드 전반의 **BidAI** → **IT'S MINE** (대문자).

9.3 프로젝트 구조

앱은 파일 업로드 화면이 아니라 **내부 RFP 문서 목록 기반 분석 흐름**을 기본으로 사용합니다. 화면 코드(`views/`)는 `src.rag` 를 직접 import하지 않고 `services/` 계층을 통해 RAG 기능을 호출합니다.

```

app/
├─ app.py                Streamlit 실행 파일, 공통 session_state 초기화
├─ README.md            앱 설명 파일
├─ views/
│   ├─ home.py          홈
│   ├─ documents.py     내부 문서 검색/선택/분석 시작
│   ├─ workspace.py     분석 결과 + 대화형 탐색 진입점
│   ├─ chat.py          선택 문서 RAG 채팅 전용 화면
│   └─ pricing.py       요금제
├─ services/
│   ├─ chat_jobs.py     백그라운드 채팅 job queue
│   ├─ frontend_adapter.py  화면 코드와 RAG 서비스 사이 변환 계층
│   ├─ rag_service.py   Streamlit UI용 RAG 서비스 함수
│   └─ sqlite_store.py  run/document/chat_history 저장소
├─ examples/
│   ├─ build_internal_corpus.py 내부 문서 전체 ingest 예시
│   └─ list_corpus_runs.py   생성된 corpus run 확인
├─ utils/
│   ├─ styles.py         전역 CSS
│   ├─ components.py     상단 네비게이션/공통 컴포넌트
│   └─ mock_data.py      RAG 미연결 로컬 미리보기 데이터
└─ .streamlit/
    └─ config.toml       Streamlit 테마 설정

```

9.4 파일별 구현 내용

분석 흐름(`views/`): 기본 화면은 내부 네비게이션을 통해 `documents.py` 로 이동해 문서를 선택하고, 분석 완료 후 `workspace.py` 에서 결과를 확인한 뒤 `chat.py` 에서 선택 문서 범위 RAG 채팅을 수행하는 흐름입니다. `documents.py` 는 `internal_corpus()` 로 가장 적합한 corpus run과 문서 목록을 조회하고, 사용자가 문서 1건을 선택하면 `analyze_selection()` 이 `summarize()` 와 `extract_requirements()` 를 실행합니다. `workspace.py` 의 **사업 개요** 탭은 문서에서 확인된 값만 보여주고 **명시되지 않음** 같은 미확인 값은 화면에서 숨깁니다.

서비스 계층(`services/`): 화면과 RAG 사이의 변환·실행을 담당합니다. `frontend_adapter.py` 가 화면 코드와 RAG 서비스 사이의 변환 계층 역할을 하고, `rag_service.py` 가 Streamlit UI용 RAG 함수를 제공합니다. `chat.py` 의 질의는 `chat_jobs.py` 의 백그라운드 job에서 `ask_with_document_filter(run_id, question, selected_doc_ids)` 를 호출해 선택 문서 범위를 전달합니다. `run/document/chat_history` 는 `sqlite_store.py` 가 저장·관리하며, UI는 응답의 `reply` , `structured_output` , `citations` 를 분리해 표시합니다.

내부 corpus 준비(`examples/`): VM에서 내부 원문 폴더를 한 번 ingest해 corpus run을 생성합니다. `build_internal_corpus.py` 로 내부 문서 전체를 인덱싱하고, `list_corpus_runs.py` 로 생성된 run을 확인합니다. corpus run과 `rag_service.db` 는 기본적으로 `/shared/experiments/streamlit/` 에 저장되며, 시연 안정성을 위해 검증된 run을 `RAG_CORPUS_RUN_ID` 로 고정하는 방식을 권장합니다.

```

python app/examples/build_internal_corpus.py --raw-docs-dir /shared/data/raw_docs
python app/examples/list_corpus_runs.py
export RAG_CORPUS_RUN_ID=<run_id>

```

공통 코드(`utils/`) 및 실행 모드: `styles.py` 가 전역 CSS를, `components.py` 가 상단 네비게이션·공통 컴포넌트를, `mock_data.py` 가 RAG 미연결 시 로컬 미리보기 데이터를 담당합니다. 실행 모드는 `RAG_MODE` 로 명시할 수 있으며, `mock` 이면 RAG 연결 없이 화면을 미리보고, `rag` 이면 연결 실패 시 상단 배너에 실패 상태를 표시하며, 미설정 시 RAG import가 가능하면 실제 RAG로, 아니면 mock으로 fallback합니다.

`RAG_EAGER_ANALYSIS=1` 이면 문서 선택 시점에 LLM 요약·요구사항 추출까지 즉시 수행하고, 미설정 시 저장된 corpus 메타데이터로 워크스페이스를 빠르게 엽니다.

RAG 연결 흐름 요약:

1. VM에서 내부 원문 폴더를 한 번 ingest해 corpus run 생성
2. documents.py가 internal_corpus()로 corpus run·문서 목록 조회
3. 사용자가 문서 1건 선택
4. analyze_selection()이 summarize() + extract_requirements() 실행
5. chat.py가 chat_jobs.py를 통해 선택 문서 범위 RAG 질의를 백그라운드 job으로 수행
6. UI는 reply · structured_output · citations를 분리해 표시

설정 메모: Streamlit config는 `configs/experiments/rag/streamlit.yaml` 을 사용하며 `agent/agent_lplus.yaml` 과 `config_final.yaml` 을 `base_config` 로 상속합니다. 앱 경로에서는 `streamlit.yaml` 이 `vector_store.type: chroma` 를 override하므로 Chroma 기반 검색을 사용합니다. 시연 추천 질문은 `사업 예산은?` , `참가 자격은?` , `제출 서류는?` 이며, `평가 기준` · `사업기간` · `제출마감` 처럼 문서마다 표기와 표 구조가 크게 다른 항목은 후보 근거 검색은 가능하나 확정값 추출·포맷팅은 고도화 과제로 둡니다.

9.5 검증

작업본을 헤드리스 Streamlit 및 `AppTest` 로 실행해 다음을 확인했습니다.

- 각 페이지(`home` / `documents` / `workspace` / `chat` / `pricing`)가 런타임 예외 없이 실행됨
- `documents.py` 가 `internal_corpus()` 로 corpus run과 문서 목록을 정상 조회함
- 문서 선택 시 `analyze_selection()` 이 `summarize()` + `extract_requirements()` 를 실행함
- `chat.py` 의 질의가 `chat_jobs.py` 백그라운드 job으로 처리되고, `reply` / `structured_output` / `citations` 가 분리 표시됨
- `RAG_MODE=mock` 에서 RAG 미연결 상태로도 화면이 정상 렌더링됨
- `RAG_CORPUS_RUN_ID` 로 고정한 corpus run이 앱에서 정상 사용됨

9.6 실행·디버깅 과정에서 겪은 이슈

작업 중 실제로 마주친 문제와 해결책입니다. 대부분 코드 버그가 아닌 환경/실행 관련 이슈였습니다.

증상	원인	해결
<code>pip ... 인식되지 않습니다</code>	pip이 PATH에 없음	<code>python -m pip ...</code> 로 실행
프롬프트가 <code>>>></code> 로 바뀜	파이썬 REPL 진입	<code>exit()</code> 로 빠져나오기
<code>No module named run</code>	<code>python -m run streamlit</code> 순서 오류	<code>python -m streamlit run app.py</code>
<code>File has no extension</code>	<code>app.py</code> (첨표 오타)	<code>app.py</code> (마침표)
상단바가 잘려 안 보임	<code>.block-container</code> 상단 여백 부족	<code>padding-top</code> 상향
<code>not enough values to unpack (expected 3, got 2)</code>	<code>st.columns</code> 칸 수와 변수 수 불일치	변수 개수 = 칸 개수로 맞춤
브랜드 글자 크기 변경 무반응	미사용 옛 <code>.brand</code> 규칙을 수정함	실제 적용되는 <code>.st-key-brandbar</code> 규칙 수정
<code>ConnectionReset [WinError 10054]</code>	앱 종료/새로고침 시 연결 끊김(정상)	무시

참고: `.gitignore` 권장 항목 — `__pycache__` / , `*.pyc` , `.streamlit/secrets.toml`

10. 백엔드 서비스 구현 및 고도화 계획

실험으로 검증된 RAG 파이프라인을 실제 서비스로 옮기는 단계입니다. 방향은 **재설계가 아니라 점진적 개선**으로 잡았습니다. 아키텍처 core(pipeline, chatbot, agent)는 유지하고, storage layer(CSV/JSONL → ChromaDB + SQLite)와 presentation layer(응답 포맷), prompt engineering만 개선합니다.

10.1 인메모리 처리 → DB 전환

현재 서비스는 챗봇 캐시·문서 컨텍스트·대화 기록·임베딩을 전부 메모리나 파일에 두어 휘발성이 큼니다. 이를 ChromaDB + SQLite로 전환합니다.

이슈	현재	변경 방향
<code>_chatbot_cache</code> dict 휘발성	rag_service.py	SQLite로 run_id별 연결 상태 관리
<code>load_document_context()</code> 전체 메모리 적재	chatbot.py	<code>connect_collection(run_id)</code> 로 대체, ChromaDB 쿼리
<code>_ChatMemory</code> dict 휘발성	pipeline.py	sqlite3 기반 <code>chat_history</code> 테이블, thread_id로 조회/저장
<code>embeddings.jsonl</code> 파일 기반	adapters/chatbot	<code>ChromaRetrieverAdapter</code> 신규 구현
run 관리 파일시스템 순회	rag_service.py	SQLite <code>runs</code> 테이블 조회
문서 목록 매번 CSV 파싱	rag_service.py	SQLite <code>documents</code> 테이블 / ChromaDB 메타데이터 조회

[변경 전]	[변경 후]
embeddings.jsonl 파일	ChromaDB Collection (영속)
↓	↓
ChatbotRunner.load_document_context()	ChatbotRunner.connect_collection(run_id)
↓	↓
self.embeddings (list, in-memory)	collection.query(query_embedding, top_k)

10.2 챗봇 응답 품질 개선

- **구조화 출력과 자연어 분리**: `structured_output` 을 자연어 reply에 섞지 않고 "요약 카드"용 마크다운 테이블로 별도 렌더링. reply가 비어있을 때만 fallback으로 노출.
- **시스템 프롬프트 페르소나화**: "JSON으로 응답하라" → "너는 RFP 입찰 전문 컨설턴트 'IT'S MINE'입니다. 사용자 질문을 이해하고 자연스러운 대화로 답변하라. 분석이 필요하면 내부 도구를 호출하라."
- **출처 접이식 렌더링**: 출처 블록을 무조건 제거하지 않고 `citations` 리스트를 UI에 전달해 접이식(foldable) 영역으로 표시.
- **추출 프롬프트 완화**: `extract_facts/extract_requirements`를 "명시된 내용만" → "근거 기반으로 자연스럽게 요약하되 명시 안 된 항목은 '명시되지 않음'으로 표시", temperature 0.1 → 0.3.

10.3 관련 없는 질문 처리 (off-topic 필터)

RFP와 무관한 질문("오늘 날씨 어때?")에 엉뚱한 검색을 돌리지 않도록 intent classifier를 도입합니다.

```
chat(user_input)
  ↓
_classify_intent(user_input) ← 신규
├─ "rfp_question" → _select_tool() → _run_agent_loop()
├─ "general"      → out_of_scope_reply 반환
└─ "feature_help" → 사용가능 기능 목록 반환
```

out-of-scope 응답: "저는 RFP 문서 전문 분석 도우미입니다. 문서 요약, 요구사항 추출, 비교 분석, 참여 판단에 대해 질문해 주세요."

10.4 응답 속도

- **SLA config**: `target_first_response_ms: 3000`, `target_full_analysis_ms: 10000`, 초과 시 로그 경고
- **질문 캐싱**: `key = hash(question + sorted(doc_ids))`, TTL 300초, 동일 질문+문서 조합이면 Tool 실행 생략
- **검색 최적화**: BM25+RRF 실제 적용 검증
- **LLM 호출 최적화**: `input_from` 이 있는 Tool은 이전 Tool의 `structured_output`을 context로 주입해 청크 재검색 생략

10.5 임베딩 용량 관리

ChromaDB 사용으로 인덱스 관리가 자동화됩니다. chunk당 $768\text{dim} \times 4\text{byte} = 3\text{KB}$ 기준, 5,000 chunks $\approx 15\text{MB}$, 100,000 chunks $\approx 300\text{MB}$ 로 디스크에 저장돼 메모리 부담이 없습니다. 컬렉션별 최대 chunk 수 제한(`max_chunks_per_run: 10000`)과 차원 동적 결정(64dim 고정 $\rightarrow$ config 기반)도 추가합니다.

10.6 요약 정제 및 서비스 완성도

- 빈 값 비율(`coverage_rate`) 계산, "명시되지 않음" 50% 초과 시 경고
 - 누락 필드 원인 명시("※ '사업예산' 항목은 문서 내에 명시되지 않았습니다")
 - `graded judge`(정확/부분정확/부정확/환각/무응답 5단계)
 - 에러 핸들링 데코레이터, 로깅 체계, `thread-safe` 세션, 비동기 `ingest`, 진행률 UI
-

11. 서비스 발전 방향

현재 서비스는 "문서를 빠르게 읽어주는 도구"에 가깝습니다. 향후에는 단순 요약을 넘어 **입찰 의사결정 지원 서비스**로 고도화하는 전략을 구상하고 있습니다. 사용자의 의사결정 흐름(어떤 공고를 볼지 고르기 → 참여 가능한지 판단 → 실제 제안 준비)에 맞춰 세 가지 서비스로 정리했습니다.

11.1 서비스 1 — 공고 탐색·비교·추천

여러 RFP 공고를 조건별로 나란히 비교하고, 우리 회사에 가장 적합한 공고를 우선순위로 추천합니다. "입찰가 1억 이상, 사업기간 6개월 이하 공고 찾아줘"라는 자연어 요청에 전체 corpus에서 조건에 맞는 후보를 검색해 예산·사업기간·참가자격·제출서류·평가기준을 표로 비교하고, 적합도 점수로 검토 우선순위를 제시합니다. 이 기능은 메타 헤더 삽입 작업 덕분에 "조건으로 검색하고 표로 비교"가 정확히 작동합니다.

11.2 서비스 2 — 참여 가능성 진단

회사 프로필(규모·인증·실적·인력)을 등록해두면 특정 공고의 참여 가능 여부와 부족한 요건을 자동 분석합니다. 여기에 리스크 탐지를 결합해, 과도한 보안 요구사항·짧은 수행기간·높은 실적 기준·공동수급 필요 여부처럼 놓치기 쉬운 위험까지 함께 표시합니다. 참여 가능성이 "요건을 충족하는가"라면 리스크는 "충족하더라도 부담스러운 조건은 없는가"로, 둘은 같은 질문의 양면입니다. 최종 결과는 "가능하지만 이런 리스크가 있음"이라는 입체적 진단이 됩니다.

11.3 서비스 3 — 제안 준비 어시스턴트

참여를 결정한 공고에 대해 제출서류·마감일·필수 자격·보증서류를 추출해 실무자가 바로 쓸 수 있는 체크리스트로 제공합니다. 마감일 정렬, 필수/선택 구분, 준비 상태 체크 형태로 만들면 실무 도구가 됩니다. 이 기능은 제출서류·자격 카테고리 검색 정확도에 크게 의존하는데, 리포트에서 제출서류 hit rate 0%를 청킹 개선으로 끌어올린 것이 이 체크리스트의 신뢰도를 좌우하는 기반이 됩니다.

11.4 세 서비스의 관계

단계	서비스	사용자의 질문
① 탐색	공고 탐색·비교·추천	"어떤 공고를 볼까?"
② 판단	참여 가능성 진단	"우리가 들어갈 수 있나?"
③ 실행	제안 준비 어시스턴트	"뭘 준비해야 하나?"

세 서비스는 따로 노는 게 아니라 하나의 의사결정 파이프라인으로 이어집니다. 이 흐름이 "요약 도구 → 입찰 의사결정 지원 플랫폼"이라는 고도화 서사를 완성합니다.

12. 한계 및 남은 과제

12.1 데이터 품질

- **HWP 잔재 문자의 근본 해결 불가**: HWP 페이지번호 인코딩이 예측 불가능한 유니코드 영역(CJK 부수 → 수학기호 → 점자 블록)에서 깨질 수 있어, 범위 기반 필터링으로 모든 경우를 잡는다는 보장이 없습니다. 이후 단계까지 발견되는 대로 패치했으나 완결되었다고 보기는 어렵습니다.
- **표 구조 미복원**: 파서는 표의 행/열 구조를 복원하지 않고 텍스트만 일렬로 풀어냅니다. RAG 검색용으로는 충분하나 "표 몇 번째 줄 값" 같은 구조 의존 질문에는 한계가 있습니다.
- **골든셋 일부 미완 검증**: v11 기준 371건 중 13건은 청크 배정은 검증됐으나 문자열 완전일치 기준만 미충족 상태로, 실험 결과로 최종 판단하기로 남겨두었습니다.

12.2 청킹·corpus 정보 결정

서비스 corpus(오버랩 80, 약 16,700 청크)와 실험 corpus(오버랩 40, 14,717 청크)의 설정이 갈려 있습니다. 어느 쪽을 정보원으로 할지 팀 결정이 필요합니다.

12.3 답변 생성 단계 병목

정형 카테고리(예산/발주기관/일정)는 청킹 개선과 질문 제목 정상화로 retrieval이 크게 개선됐습니다. 그러나 자격·제출서류는 "검색은 되지만 답변 생성 단계에서 과도하게 회피 응답(hedging)한다"는 다음 병목으로 넘어갔습니다. 이는 데이터가 아니라 answerer 프롬프트의 문제로, 향후 개선 대상입니다.

12.4 재현성

`langchain-text-splitters` 설치 여부에 따라 같은 config로도 실제 청크 경계가 달라질 수 있습니다. 팀원 간 환경을 맞추거나 결과 비교 시 이 점을 감안해야 합니다.

12.5 코드 미커밋 사항

`document_loader.py`의 메타 프리앰블 수정과 HWP 파서 수정이 미커밋 상태로, CSV ingest 전체에 영향을 주는 공용 코드입니다. 커밋 전 팀 리뷰가 필요합니다. 또한 git 워크플로 관련해 커밋이 formal PR 리뷰 없이 origin/jinwoo와 main 양쪽에 올라간 이력이 있어 팀 리더와 공유가 필요합니다.

13. 부록: 버전 계보 및 핵심 수치

13.1 data_list 버전별 변화

버전	행 수	텍스트 최솟값	텍스트 평균	결측	주요 변경
data_list.csv (초기)	100	89자	3,843자	35건	원본
(HWP 복원 후)	100	533자	~30,000자	33건	HWP 7건 복원
버전3	98	533자 ↑	—	17건	중복 2건 제거, 날짜 오류 수정
버전4	98	42,769자	72,802자	0건	91건 전체 재추출
버전5	98	—	—	0건	청킹용(목차 제거 + 전 청크 메타 프리앰블)

13.2 eval_questions 버전별 변화

버전	문항 수	chunk_id	answer 검증	비고
v1	897	없음	안 됨	LLM 의미 추출 도입
v3	200	없음	안 됨	비례 다운샘플링
v4	344	100%	안 됨	LLM 버그 수정
v6_final	378	100%	100% True	verbatim 검증 + 균형화
v7_final	378	100%	재매핑	버전5 청크 기준
v8	378	fuzzy 42건 수정	미검증	라벨 교정
v9	378	재청킹 재매핑	미검증	overlap 40
v10	377	유지	콘텐츠 37건 수정	중복 병합
v11_final	371	유지	358 완전 + 13 청크만	콘텐츠 정밀 검증
v12_final	370	예산 라벨 통일	—	질문 제목 26건 수정 (최신)

13.3 청크 수 변화

시점	청크 수	비고
버전4 기반	16,218~16,222개	overlap 80, 메타 프리앰블 chunk_0001만
버전5	14,717개	overlap 40, 전 청크 메타 헤더, 목차 88문서 제거

13.4 카테고리별 retrieval hit rate 변화

시점	예산	발주기관	일정	자격	제출서류
v6 (hashing, mini50)	0%	8%	0%	14%	0%
청킹 개편 직전(v6_final, 378)	37.4%	29.9%	8.4%	10.4%	0.0%
v12 재실험	100%	100%	100%	개선(병목: hedging)	개선(병목: hedging)

13.5 최종 실험 설정 요약

- 임베딩: OpenAI `text-embedding-3-small`
- 답변 모델: OpenAI `gpt-5-mini`, temperature 0.0
- retrieval: top_k=15 + reranker(BAAI/bge-reranker-v2-m3) + BM25 hybrid(RRF)
- 데이터: v12 (370문항 골든셋, 14,717 청크)
- 최종 성능(v12mini top15+reranker): retrieval_hit 96%, hit@1 80%, citation 92%, judge_correct 80%, retrieval_failure 4%
- Vector DB: ChromaDB (확장 시 PostgreSQL+pgvector)